



НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ

Системный дизайн современных приложений

Занятие №7
Базы данных. Часть 2.
Распределенные системы



Репликация, шардирование, партиционирование



Horizontal scaling



Vertical scaling



Partition Strategies

Customer ID	First name	Last name	Favorite color
1	TAEKO	OHNASHI	blue
2	OLV	WRIGHT	green
3	SELDA	BAGGAN	purple
4	JIM	PEPPER	indigo

Vertical Partitions

Customer ID	First name	Last name	Favorite color
1	TAEKO	OHNASHI	blue
2	OLV	WRIGHT	green
3	SELDA	BAGGAN	purple
4	JIM	PEPPER	indigo

Customer ID	First name	Last name	Favorite color
1	TAEKO	OHNASHI	blue
2	OLV	WRIGHT	green
3	SELDA	BAGGAN	purple
4	JIM	PEPPER	indigo

Horizontal Partitions

Customer ID	First name	Last name	Favorite color
1	TAEKO	OHNASHI	blue
2	OLV	WRIGHT	green

Customer ID	First name	Last name	Favorite color
3	SELDA	BAGGAN	purple
4	JIM	PEPPER	indigo

Even distribution

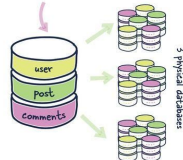


Uneven distribution



architecture notes

Monolith



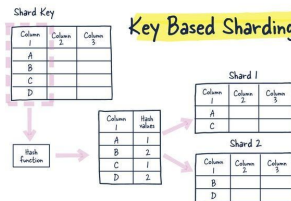
Range Based Sharding

Product	Price
WIDGET	\$100
GIZMO	\$80
TRINKET	\$75
THINGAMAJIG	\$90
DOODAD	\$60
TOOTCHKE	\$99

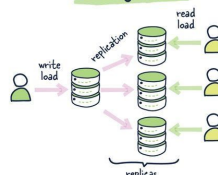
Product	Price
WIDGET	\$100
GIZMO	\$80
TRINKET	\$75
THINGAMAJIG	\$90
DOODAD	\$60
TOOTCHKE	\$99

Product	Price
WIDGET	\$100
GIZMO	\$80
TRINKET	\$75
THINGAMAJIG	\$90
DOODAD	\$60
TOOTCHKE	\$99

Key Based Sharding



Scaling Reads



Что?

Процесс копирования и синхронизации данных между несколькими узлами для обеспечения их доступности, надежности и высокой производительности

Виды

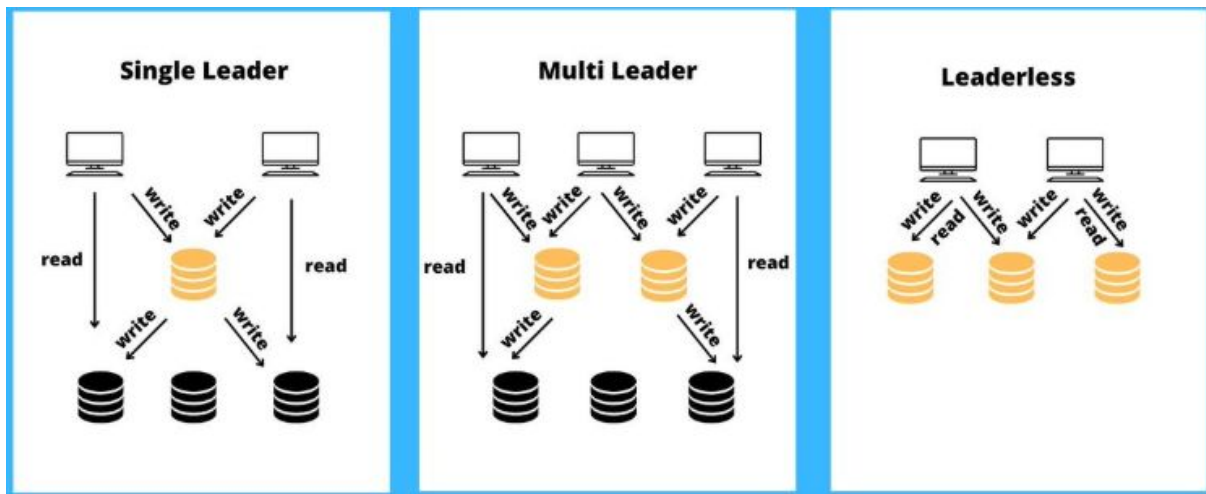
1. Single-Leader
2. Multi-Leader
3. Leaderless

Зачем создали?

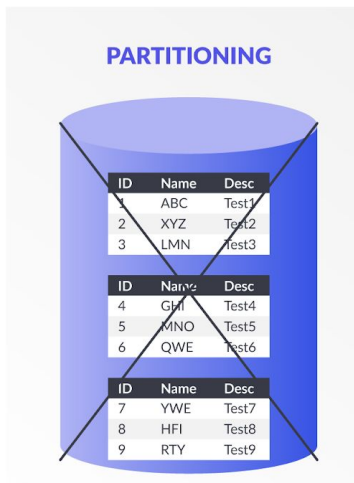
1. Повышение доступности
2. Улучшение производительности
3. Гео-распределение

Типы

1. Синхронная / Асинхронная И (в зависимости от контекста)
1. Логическая / Физическая



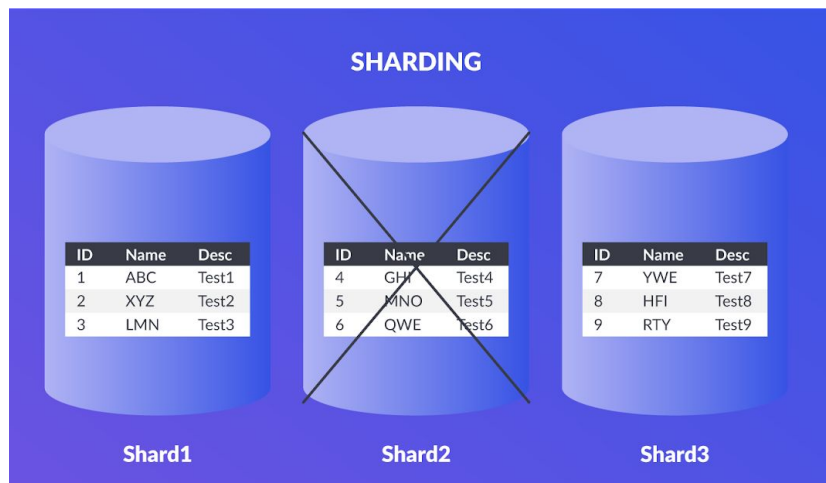
Шардинг и партиционирование



Партиционирование (Partitioning)

Логическое разделение данных внутри одной БД (например, по датам).

Цель: Ускорить запросы и упростить управление данными.

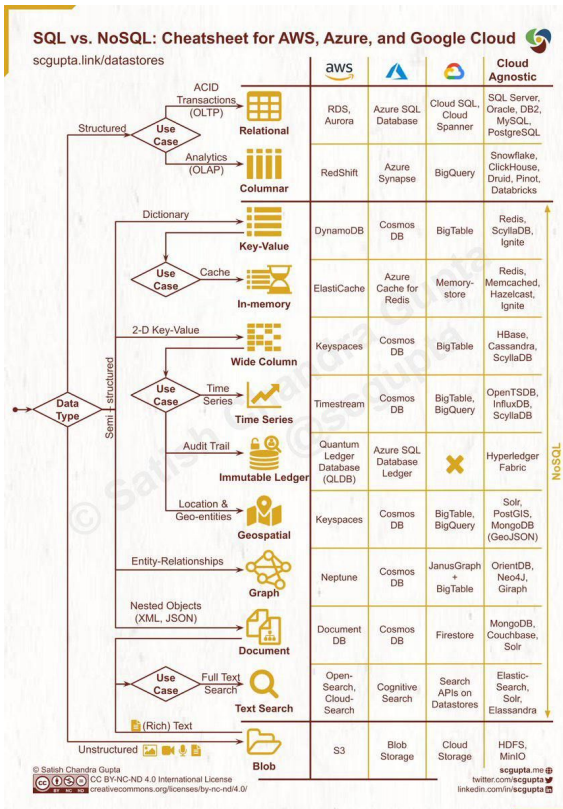


Шардинг (Sharding)

Физическое разделение данных между разными серверами (например, по user_id).

Цель: Распределить нагрузку на запись и хранить большие объемы данных.

Алгоритм выбора БД



Ряд вопросов

1. Что за данные?
2. Какой формат данных?
3. Какие отношения между данными? (ERD)
4. Какие сценарии использования данных?
5. Типовой ли проект?
6. Какие функции доступны из коробки?
7. Насколько популярна?
8. Какая лицензия?
9. Легко сопровождать и настраивать?
10. Что если данные изменятся?
11. (*) Есть ли сертификация/регистрация в ЕРРП?

<https://docs.google.com/viewerng/viewer?url=https://datafinder.ru/files/Cheat-Sheet-SUBD.pdf>



Кейс: Фото-приложение

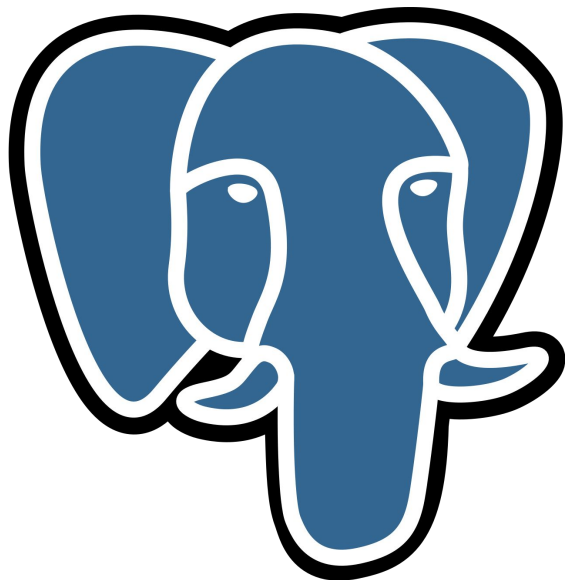


Ряд вопросов

1. Что за данные?
 - Метаданные: юзеры, био, лайки, комментарии
 - Лента и социальный граф
 - Медиафайлы (фото)
2. Какой формат данных?
 - Метаданные: структурированный, схема стабильна
 - Лента: может быть идентификаторы связей
 - Медиафайлы: бинарные объекты
3. Какие отношения между данными? (ERD)
 - Метаданные: классика
 - Лента: графовые
 - Медиа: нет внутренних отношений
4. Какие сценарии использования данных?
 - Метаданные: OLTP (CRUD), со строгим ACID
 - Лента: найти всех, на кого подписан пользователь и взять их посты. Важна скорость.
 - Медиа: загрузить/скачать
5. Типовой ли проект?
 - Метаданные: классика
 - Лента: необычно
 - Медиа: типовой для медиа
6. Какие функции доступны из коробки?
7. Насколько популярна?
8. Какая лицензия?
9. Легко сопровождать и настраивать?
10. Что если данные изменятся?
11. (*) Есть ли сертификация/регистрация в ЕРРП?

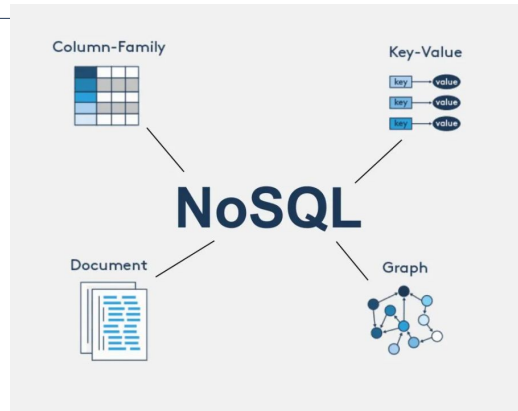
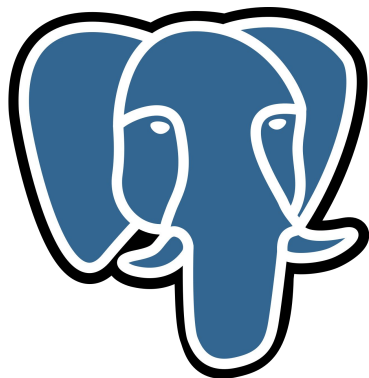
<https://docs.google.com/viewerng/viewer?url=https://datafinder.ru/files/Cheat-Sheet-SUBD.pdf>

Кейс: Фото-приложение (ранний этап)

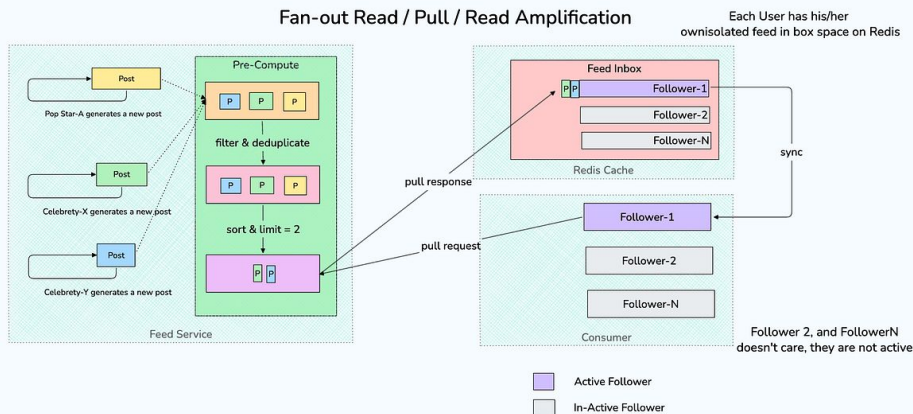




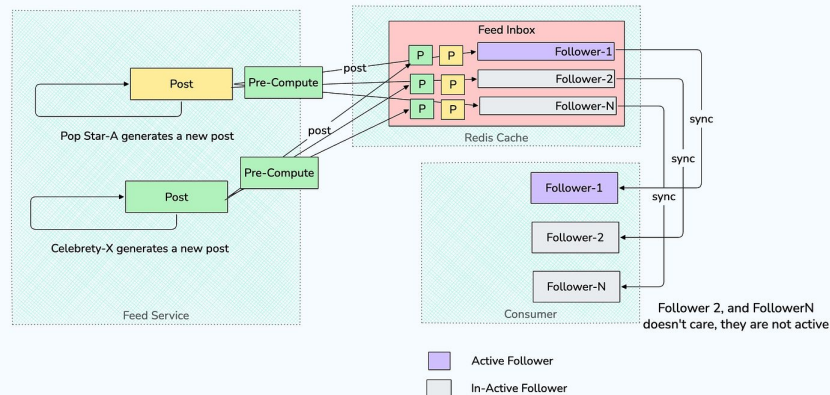
Кейс: Фото-приложение (с ростом DAU)



Fan-out Read / Pull / Read Amplification



Fan-out write / Push / Write Amplification





Распределенная система

Что?

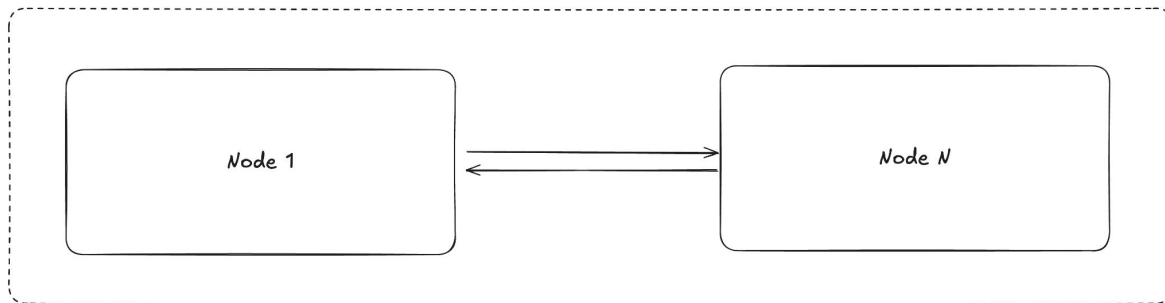
Распределенной вычислительной системой можно назвать такую систему, в которой отказ компьютера, о существовании которого вы даже не подозревали, может сделать ваш собственный компьютер непригодным к использованию

- Лесли Лампорт

Распределенная вычислительная система (РВС) – это набор соединенных каналами связи независимых компьютеров, которые с точки зрения пользователя некоторого программного обеспечения выглядят единым целым

- Эндрю Таненбаум

cluster





Почему распределенные системы победили

Ограничения single node

- + Физические ограничения железа
- + Масштабирование, отказоустойчивость, доступность
- + Один сервер
- + Объемы данных на одном диске

Сеть

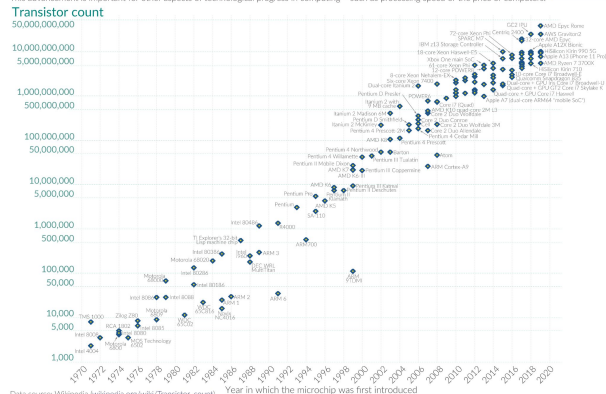
- + Стала быстрее
- + Часто вызов по сети получается сравним с локальным вызовом
- + Нужно обслуживать миллионы соединений

Cloud Native

- + Контейнеры
- + Микросервисы
- + Оркестрация

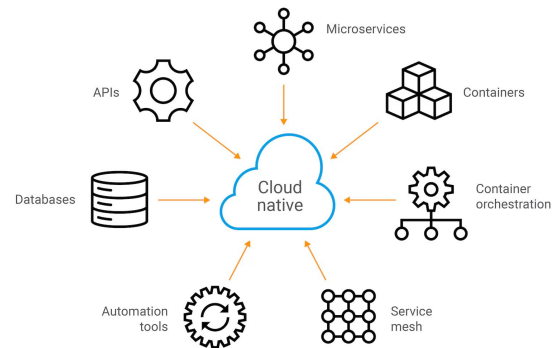
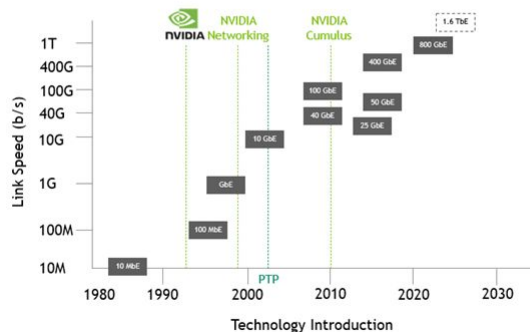
Moore's Law: The number of transistors on microchips doubles every two years. This advancement is important for other aspects of technological progress in computing – such as processing speed or the price of computers.

Our World in Data



Data source: Wikipedia (wikipedia.org/wiki/Transistor_count)
OurWorldInData.org – Research and data to make progress against the world's largest problems.
Licensed under CC-BY by the authors Hannah Ritchie and Max Roser.

ETHERNET SPEEDS OVER THE PAST 40 YEARS





Компромиссы распределенных систем

Корректность данных

- + Строгая согласованность упрощает модель данных, но повышает стоимость записи и чувствительность к сетевым задержкам
- + Кэширование и согласованность данных: быстрый ответ может быть устаревшим

Масштаб

- + Глобальная репликация и операционная сложность
- + Шардирование и сложность запросов

Производительность

- + Синхронные подтверждения и пропускная способность: надежность каждой записи против RPS/TPS/QPS
- + Оптимизация чтения и оптимизация записи: LSM-tree / B-tree нельзя выиграть на обоих фронтах

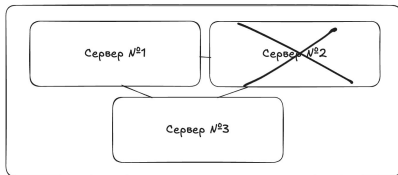


Проблемы распределенных систем

Частичные отказы

- + В монолите либо работает, либо нет
- + Появляются промежуточные состояния: вроде работает, а вроде не работает

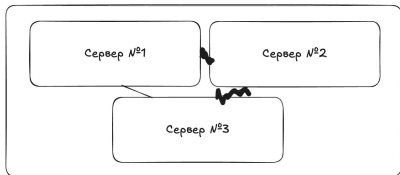
Кластер



Ненадежная сеть

- + В монолите вызов функции занимает нс и не теряется
- + Появляются сетевые вызовы: занимают мс/с, пакеты теряются/задерживаются

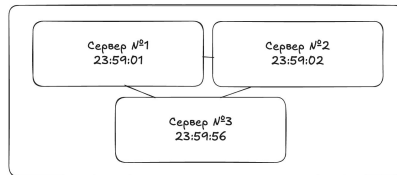
Кластер



Отсутствие глобального времени

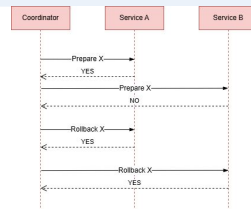
- + Каждый сервер живет по своим часам и они могут разойтись
- + Есть NTP, но может иногда недостаточно

Кластер



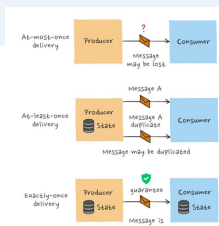
Распределенные транзакции

- + В одной базе: BEGIN ... COMMIT ROLLBACK
- + В распределенной: попробуй закоммить... Решения: 2PC, Saga, Transaction Outbox



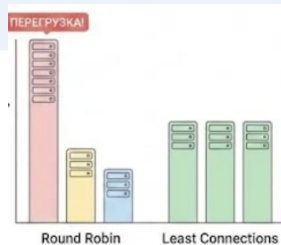
Идемпотентность

- + Гарантии доставки: at-least-once, exactly-once, at-most-once
- + Ключ идемпотентности



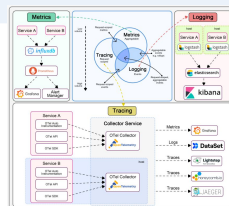
Горячие узлы

- + Неравномерная балансировка
- + Неравномерное шардирование



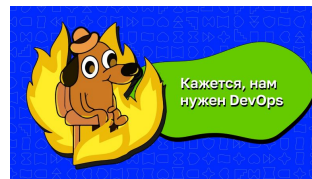
Отладка и наблюдаемость

- + В монолите - stack trace
- + В распределенной системе: а где логи? где упал запрос? Система "чуть-чуть сломалась"



Операционная сложность

- + В монолите: накатили сервис
- + В распределенной: оркестрация зависимостей, канареечные релизы, service mesh, синхронизация ...



Что?

Узлы связаны между собой, распределенная система, для клиентов - одно целое

Основная задача?

Контроль и координация

Должен быть компонент, который решает задачи:

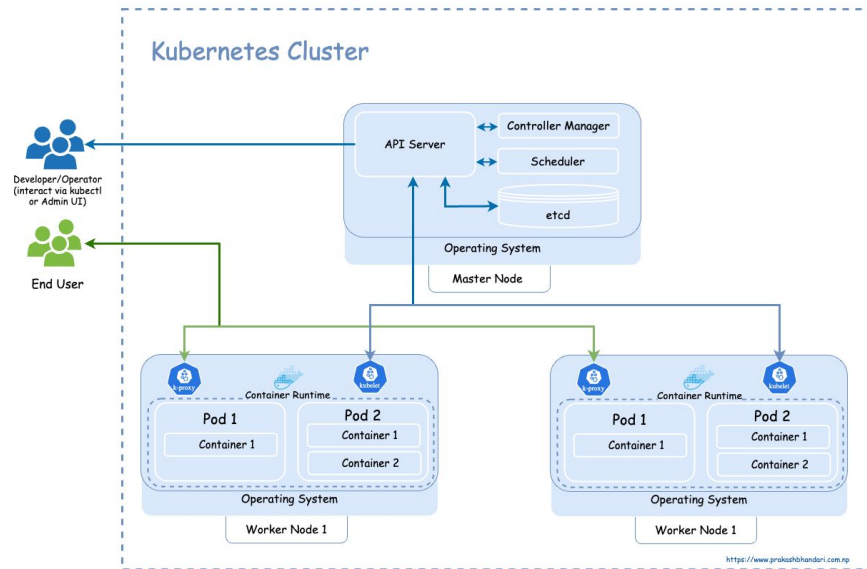
1. Хранение информации о составе кластера
2. Управление конфигурацией узлов
3. Детекция сбойных узлов
4. Выбор лидера
5. Service discovery

Инструменты

- + etcd: KV-хранилище, исп. в Kubernetes. Raft, строгая согласованность, простой API
- + Consul: service discovery + health checks + KV
- + ZooKeeper: ветеран - основа старых версий Kafka и Hadoop. Протокол ZAB, надежный, но тяжелый

Алгоритмы

- + Paxos: математически строгий, сложный в реализации
- + Raft: "Paxos для людей". Лидер-based, читаемый. Используют etcd, CockroachDB
- + Accord: новый (Cassandra 5+). Leaderless, низкая задержка, оптимизирован для мульти-регион



Детекция сбойных узлов

Heartbeat

Отправка “я живой”. Есть координатор: проблема с отказоустойчивостью и масштабируемостью

Top 6 Heartbeat Detection Mechanisms

ByteByteGo

Push-Based Heartbeat

- simple
- network congestion or delays can lead to false positives



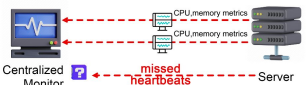
Pull-Based Heartbeat

- reduce traffic
- may increase latency in failure detection



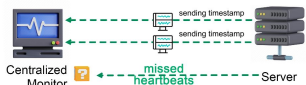
Heartbeat with Health Check

- provide details
- increase complexity and network overhead



Heartbeat with Timestamps

- detect network latency
- require synchronized clocks



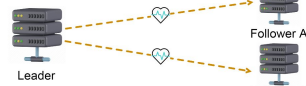
Heartbeat with Acknowledgement

- confirm sender status
- double the number of messages



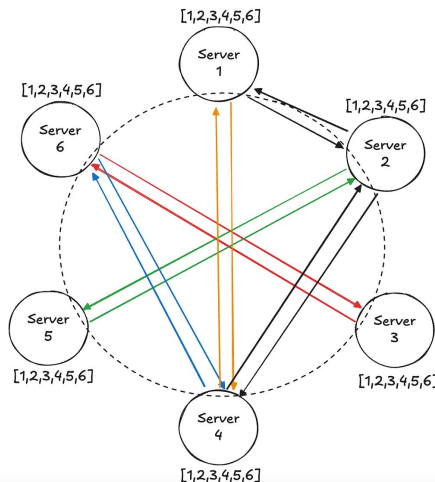
Heartbeat with Quorum

- maintain system integrity
- difficult to manage nodes



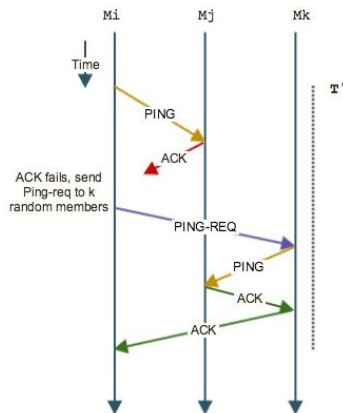
Gossip

Нет координатора. Каждый узел периодически выбирает нескольких случайных соседей и обменивается с ними своим представлением о состоянии кластера



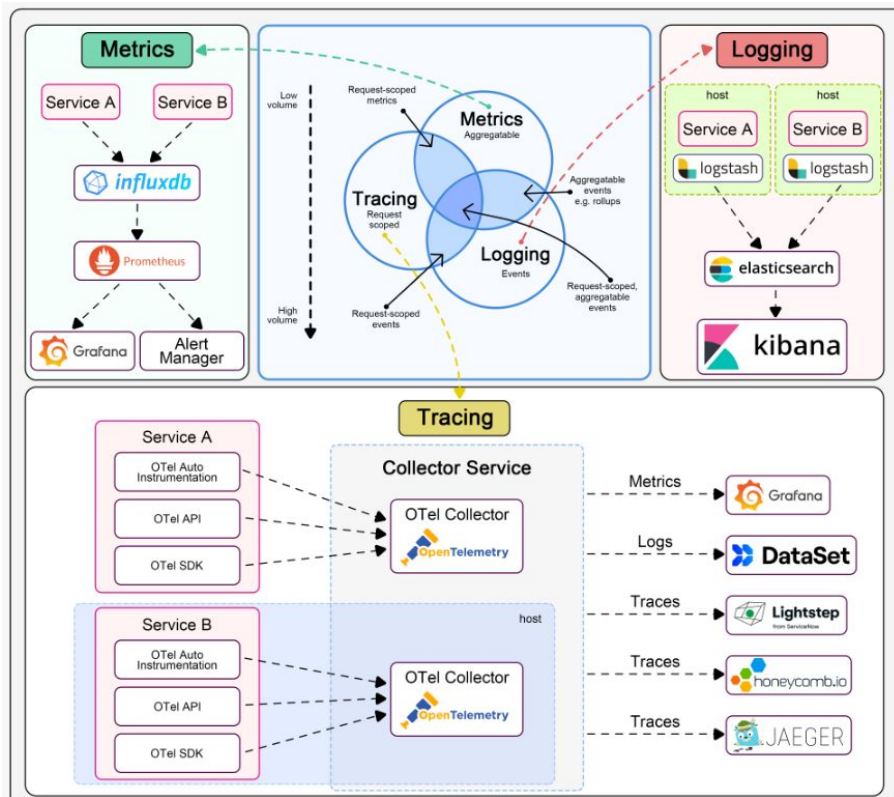
SWIM

Улучшенный Gossip





Логи, метрики, трейсы, +1



Логи

- + Что случилось?
- + Нужна централизованная агрегация
- + ELK, Grafana + Loki, Vector, в качестве пушеров всякие filebeat, fluentbit

Метрики

- + Насколько плохо, как давно?
- + Числовые измерения
- + RED, 4 gold signals
- + Prometheus + Grafana, VictoriaMetrics

Трейсы

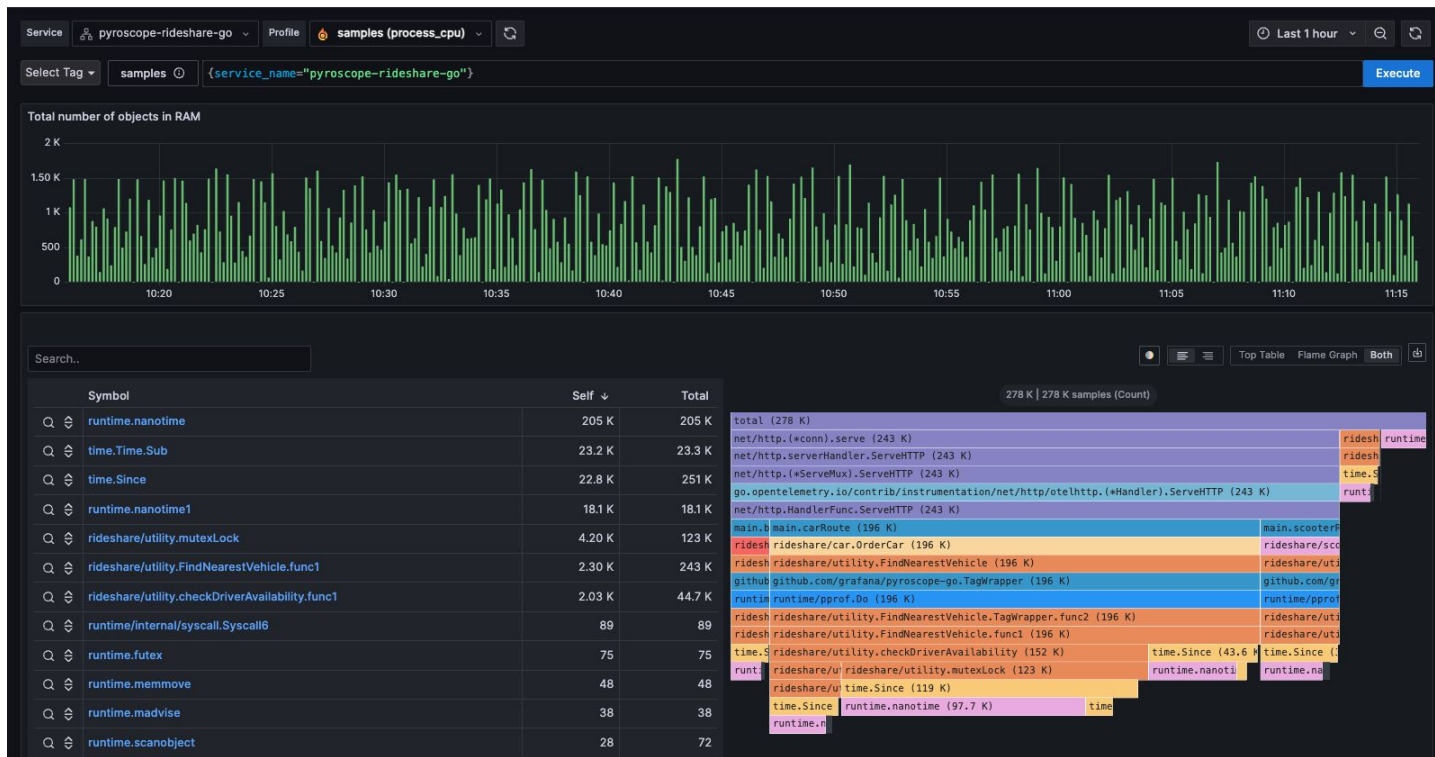
- + Где по системе оборвался/затупил запрос?
- + Сервисы создают span с общим trace_id
- + Jaeger, Zipkin

Профилирование

- + Где в коде?
- + Continuous Profiling - агент постоянно снимает профиль с каждого узла, данные централизуются
- + Pyroscope



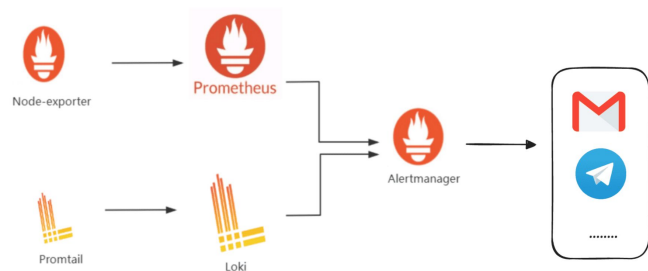
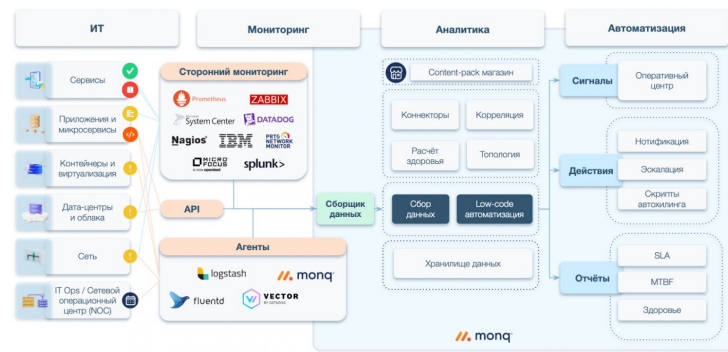
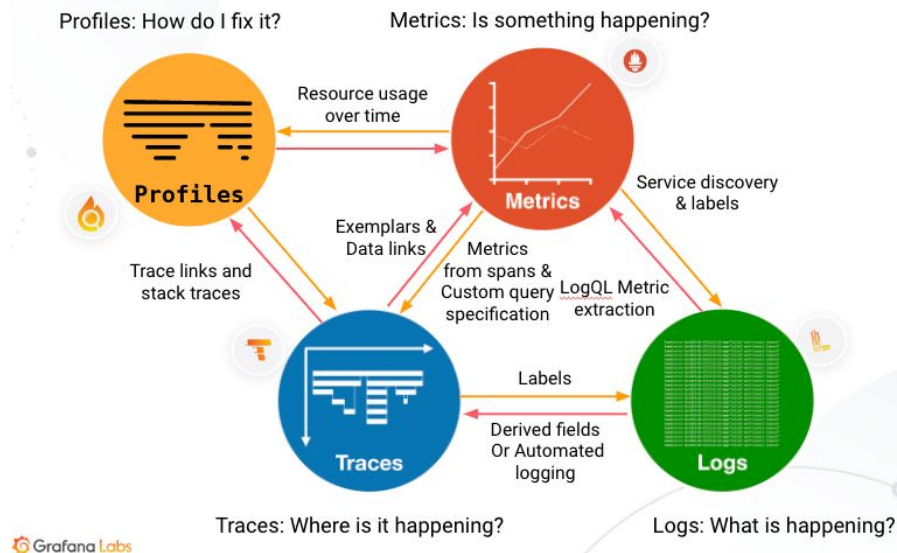
Continuous Profiling



Комбинация инструментов

Что?

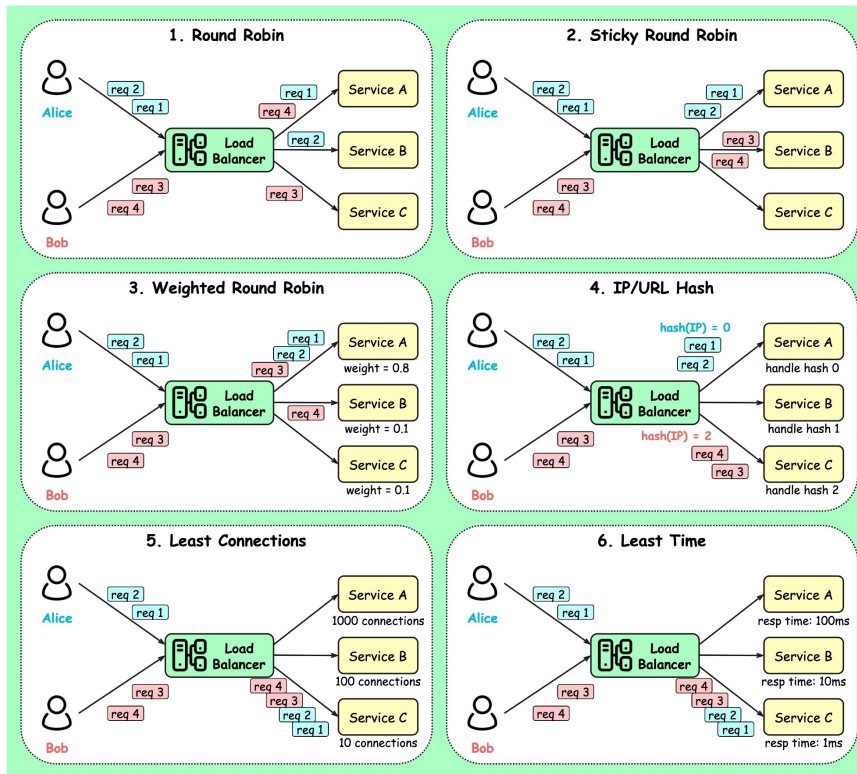
Зонтичный мониторинг: системы мониторинга систем мониторинга



Алгоритмы балансировки

Какие есть?

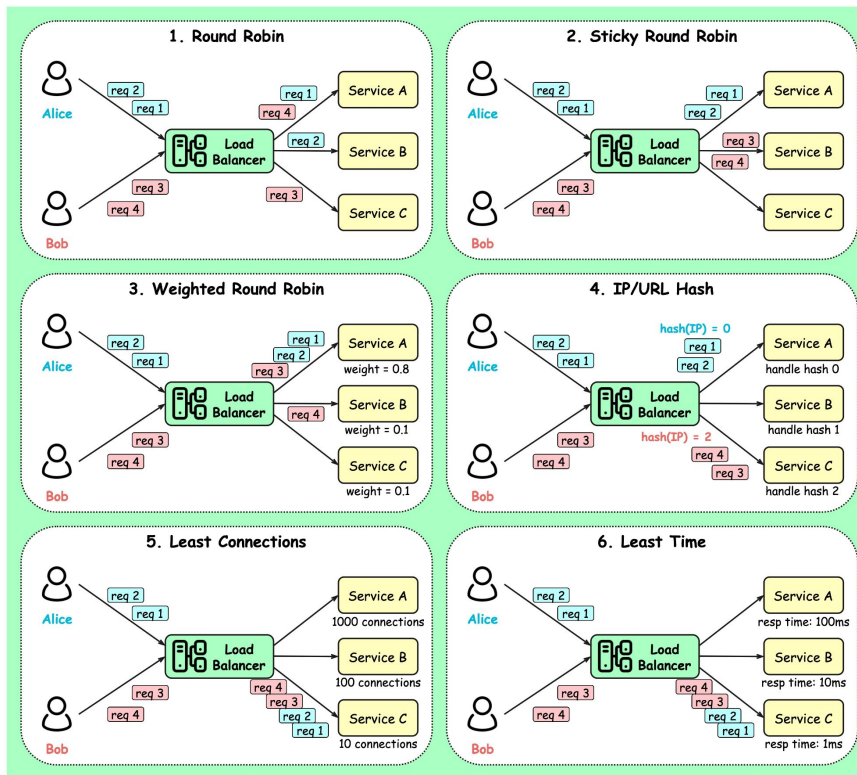
- Round Robin: когда запросы гомогенны и сервера идентичны. Ломается, когда запросы разные по весу: один запрос выполняется за 1 мс, другой за 5 секунд
- WRR: расширение для разнородного железа. Если один сервер вдвое мощнее, он получает вдвое больше запросов. Но по-прежнему не учитывает текущую загрузку
- Leastconn: отправляет новый запрос на сервер с наименьшим числом активных соединений. Ограничение: число соединений не всегда равно нагрузке
- Sticky: один и тот же клиент всегда попадает на один и тот же сервер. Неотказоустойчивый
- Least time / Least Response time: отправляет запрос на сервер с наименьшим средним временем ответа. Более точная мера нагрузки, чем число соединений. Требуется постоянное измерение - балансировщик следит за P99 каждого сервера и актуализирует рейтинг
- Hash: запрос всегда идет на один и тот же сервер, определяемый хэшем ключа



Алгоритмы балансировки

Какие есть?

- Round Robin: когда запросы гомогенны и сервера идентичны. Ломается, когда запросы разные по весу: один запрос выполняется за 1 мс, другой за 5 секунд
- WRR: расширение для разнородного железа. Если один сервер вдвое мощнее, он получает вдвое больше запросов. Но по-прежнему не учитывает текущую загрузку
- Leastconn: отправляет новый запрос на сервер с наименьшим числом активных соединений. Ограничение: число соединений не всегда равно нагрузке
- Sticky: один и тот же клиент всегда попадает на один и тот же сервер. Неотказоустойчивый
- Least time / Least Response time: отправляет запрос на сервер с наименьшим средним временем ответа. Более точная мера нагрузки, чем число соединений. Требуется постоянное измерение - балансировщик следит за P99 каждого сервера и актуализирует рейтинг
- Hash: запрос всегда идет на один и тот же сервер, определяемый хэшем ключа





НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ